基于软硬件协同设计的
个性化数字示波器实现

1 实验任务

以 2~3 人为一组，自行组队，在给定例程的基础上，完成基于软硬件协同设计的个性化数字示波器的实现。任务要求如下：

（一）基础要求

a. 实现按键消抖功能；（10 分）
b. 实现周期信号的幅度、频率测量功能，在屏幕上显示测量结果；（10 分）
c. 编码风格和代码规范性。（10 分）
   参见 https://www.runoob.com/w3cnote/verilog2-codestyle.html
   以及 https://www.runoob.com/w3cnote/verilog2-codeguide.html
d. 在示波器例程“09_Ti60F225_HDMI_720P_ADCWord_NewVerison” 单通道显示的基础上，实现双通道信号的同时显示及测量；（20 分）
e. 在示波器例程“09_Ti60F225_HDMI_720P_ADCWord_NewVerison”720p 显示的基础上，修改实现 1080p 显示；（20 分）
f. 阅读理解例程“V7_Ti60F225_RISCV_APB3_Picture”，并根据指定的系统框图重新优化代码结构；（15 分）
g. 整合上述两个例程，实现上位机实时传图，在示波器屏幕左上角显示学校 logo 或其他个性化图片（10 分）；在示波器屏幕顶部显示个性化文字（5 分）。

（二）拓展要求

自由发挥，实现创新创意功能，例如但不限于以下功能：

a. 通过按键，调整能够采样并完整显示波形的信号频率范围；
b. 显示 0V、横纵轴、刻度等；
c. 将 AD 采样数据通过 soc 串口或其他方式发送至上位机实现远程示波器；
d. ……

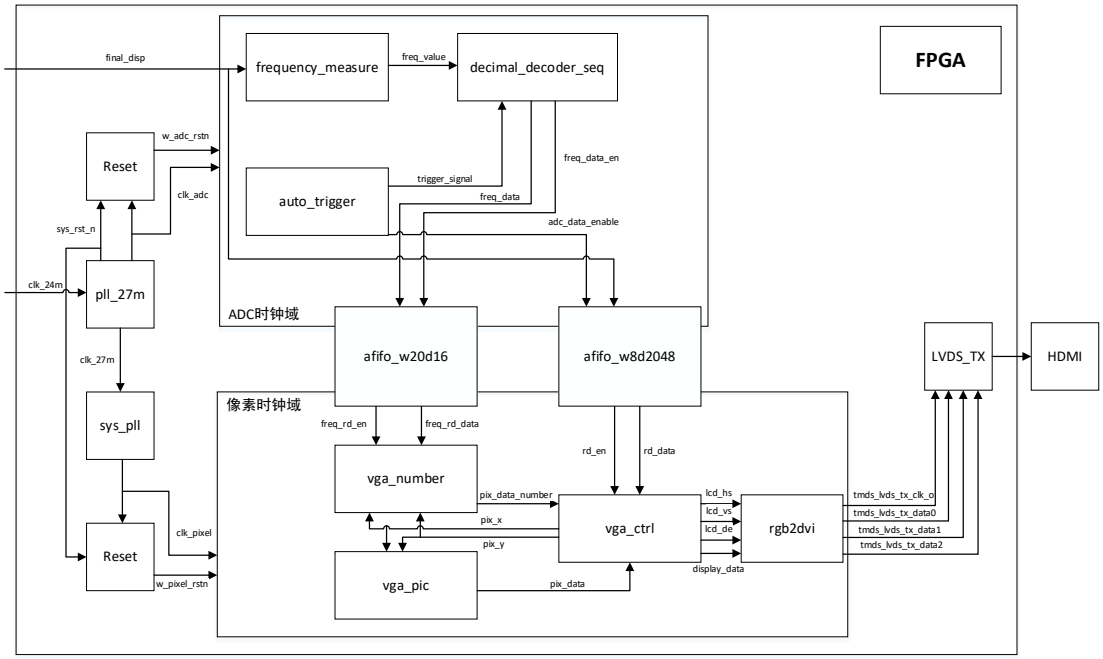
每项功能加 10 分。

2 例程介绍及模块功能说明

2.1 示波器例程介绍

工程“09_Ti60F225_HDMI_720P_ADCWord_NewVerison”实现了一个示波器系统，主要功能是将 ADC 采集并转换得到的数字信号通过 HDMI 显示波形在屏幕上，同时测量并显示信号的频率。

例程的系统框图如下所示：



（1） 顶层模块：wave_display。

（2） 按键消抖模块：key_filter，对按键输入 Key 进行去抖处理，生成按键按下标志 Key_P_Flag 信号（高电平脉冲，持续 1 个时钟周期）。要求自己实现。

（3） 复位信号跨时钟域处理模块：Reset，为 ADC 时钟域 clk_adc 和像素时钟域 clk_pixel 分别生成同步复位信号 w_adc_rstn 和 w_pixel_rstn，避免直接使用系统时钟域下的复位信号导致的跨时钟域问题。

（4） 频率测量模块：frequency_measure，测量输入信号 adc_data 的频率，输出 32 位频率测量值 freq_value（单位 kHz）。要求自己实现。

（5） 数制转换模块：decimal_decoder_seq，在 start 信号的触发下，将输入的 32 位二进制数 binary_input 转换为十进制各位（个位、十位等），供显示模块使用，转换完成后给出 done 信号。

（6） 异步 FIFO：读写时钟不同称为“异步”。例程使用了两个异步 FIFO，分别是“afifo_w20d16”和“afifo_w8d2048”。afifo_w20d16 用于将频率测量值 freq_data 从 ADC 时钟域传递到像素时钟域供显示模块使用，写入使能为数值转换模块给出的 done 信号，读出使能为 fifo 非空信号~empty_o；afifo_w8d2048 用于将 ADC 数据 final_disp 从 ADC 时钟域传递到像素时钟域供显示模块使用，写入使能为触发控制模块给出的 enable 信号，读出使能由 fifo 中现有数据个数 rd_datacount_o 以及已经从 fifo 中读取的数据个数 rd_cnt 决定。

（7） 触发控制模块：auto_trigger，每固定时间（由参数 COUNT_MAX1s 决定）输出一个 AD 时钟周期高电平的触发信号 flag_1s，作为数制转换模块的 start 信号输入；输出 enable 信号，由 flag_1s 信号触发，持续 ENABLE_COUNT 个 AD 时钟的高电平，每次触发启动 ENABLE_COUNT 个点的采样（由显示分辨率横轴宽度决定），控制 ADC 数据写入异步 FIFO（afifo_w8d2048）的时机。

（8） 视频驱动模块：vga_ctrl，生成 HDMI 时序信号（lcd_hs、lcd_vs、lcd_de），并将 ADC 数据 rd_data、文字像素 pix_data 和数字像素 pix_data_number 分别整合为波形（wave_displaydata）、文字（word_displaydata）和频率数据（number_displaydata）显示。

（9） 文字像素生成模块：vga_pic。

（10） 频率数据像素生成模块：vga_number，根据输入的频率数据 freq_rd_data，生成像素点对应的色彩信息 pix_data_number。

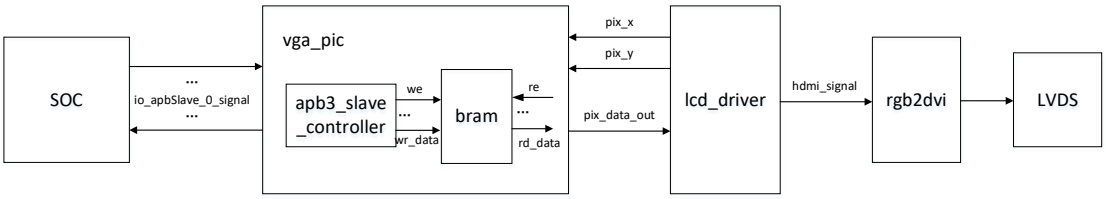
（11） rgb2dvi、tmds_channel 与《05-基于 Ti60F225 的 HDMI 显示驱动设计》中一致。

（12） PLL：例程配置了两个 PLL。pll_27m 用于产生 40MHz 的 ADC 时钟 clk_adc 和 sys_pll 的输入 27MHz 时钟 clk27m；sys_pll 用于产生像素时钟 clk_pixel。

完成按键消抖模块与频率测量模块的实现后，采集 100kHz，4Vpp 的正弦波，例程运行效果如图所示（按下 AD 模块的按键 K4 可实现通道切换）：

2.2 基于 RISC-V APB3 的图片显示例程介绍

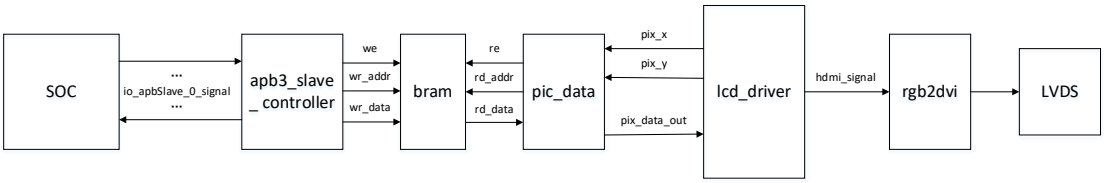
例程“V7_Ti60F225_RISCV_APB3_Picture”实现了一个基于 RISC-V APB3 实时传图并由 HDMI 显示的系统。



（1） 顶层模块：LED_8bit_Test。

（2） 图片像素生成模块：vga_pic，包含 APB3 控制器，bram 和像素生成三部分。通过 APB3 从 soc 传输图片数据到 fpga 后，以 apb 时钟域（96M）写入 bram，以像素时钟域从 bram 中读出，供显示使用。

基础要求 f：按照如下框图优化原工程代码结构，将 vga_pic、apb3 slave controller、bram 三者分离。



（3） HDMI 驱动模块：lcd_driver，生成 HDMI 时序信号。

（4） rgb2dvi，tmds_channel：与前述一致。

例程运行效果如图所示：

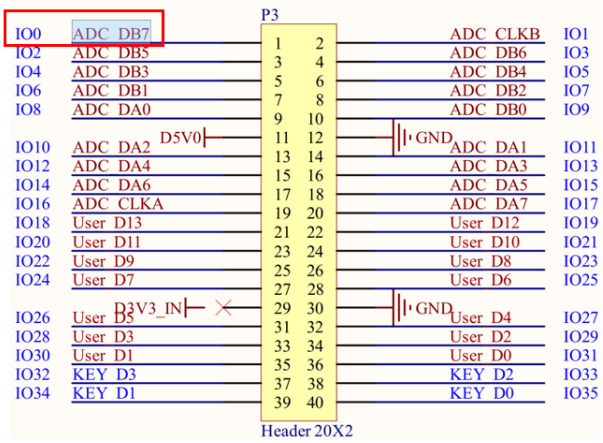
3 ADC 使用及接口说明

将 AD9288 模块与 FPGA 按如图所示相接。

打开 AD9288 的原理图，可找到排母 P3。观察 AD 模块实物，方形焊盘代表 1 号焊盘，正好与 FPGA 排插 P4 的 1 号焊盘对应。

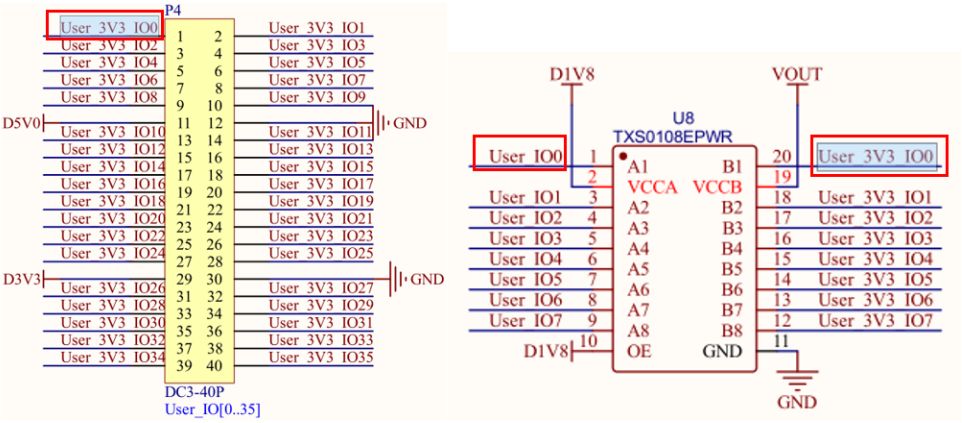
ADC_DA7~DA0 对应 INA 转换的 8 位数字信号，对应代码中[7:0] DAM；DB7~ DB0 对应 INB 转换的 8 位数字信号，对应代码中[7:0] DBM。INA，INB 的输入电压幅值范围为±5V。ADC_CLKA，ADC_CLKB 分别为 INA，INB 的采样时钟，时钟频率范围为 10M~100MHz。

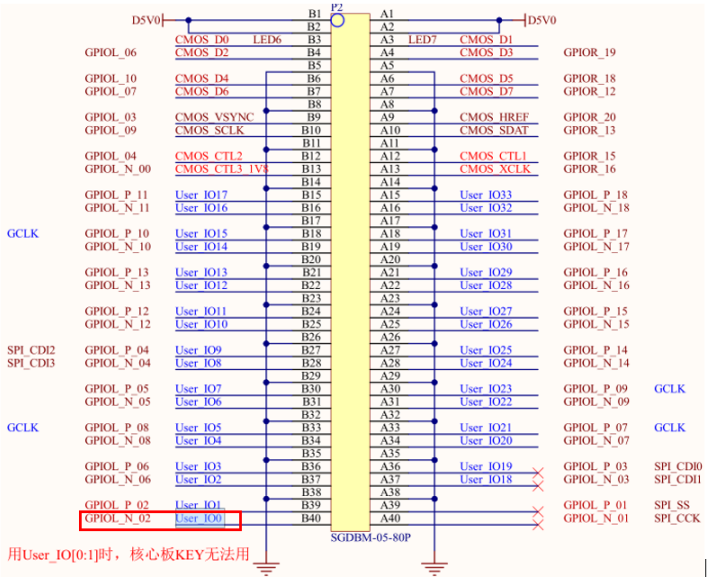
以 ADC_DB7 为例，其位于 AD 模块排母 P3 的 1 号管脚，对应 FPGA 排插 P4 的 1 号管脚。



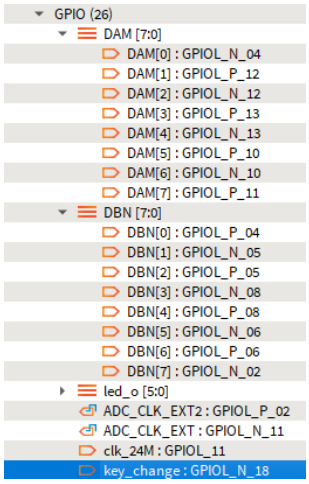
打开 FPGA 底板的原理图，找到排针 P4，搜索 1 号管脚的名字 User_3V3_IO0，其先经过电平转换芯片 U8 后与 1.8V 电平的 User_IO0 相连。继续检索，最终找到其与 GPIOL_N_02

相连。按照此方法，最终找到所有接口与 FPGA 绑定的管脚。





打开 Efinity 2024.2，打开工程“09_Ti60F225_HDMI_720P_ADCWord_NewVerison”，进入 Interface Designer 界面，可见管脚绑定如图所示。I/O Standard 的设置涉及 BANK 检索，可参照文档《02-基于 Ti60F225 的 8 位 LED 流水灯驱动设计》进行。其中 key_change 与 GPIOL_N_18 绑定，通过原理图检索其为 AD 模块的按键 K4。



4 信号跨时钟域处理

4.1 理解亚稳态

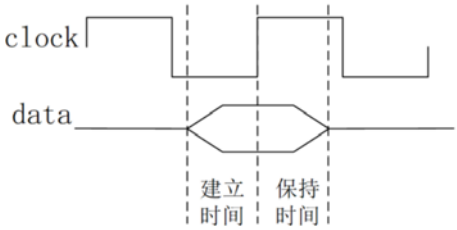
4.1.1 什么是亚稳态

对于数字系统而言，建立时间（setup time）和保持时间（hold time）是数字电路时序的基础。数字电路系统的稳定性，基本取决于时序是否满足建立时间和保持时间。

建立时间 Tsu，是指在时钟的有效沿（上升沿）到来之前，触发器的输入端信号必须保持稳定的最短时间，以便数据能够被时钟正确的采样；

保持时间 Thd，是指在时钟的有效沿（上升沿）到来之后，触发器的输入端信号必须保持稳定的最短时间，以便数据能够被电路准确的传输。

可以通俗的理解为：时钟到来之前，数据需要提前准备好；时钟到来之后，数据还要稳定一段时间。建立时间和保持时间组成了数据稳定的窗口，如下图所示。



如果数据输入信号在这段时期发生了变化，那么输出将可能是未知的或者称为“亚稳的”。这种有害状态的传播就叫做**亚稳态**。触发器的输出会因此而产生毛刺，或者暂时保持在不稳定状态而且需要较长时间才能回到稳定状态。当触发器处在亚稳态时，输出会在高低电平之间波动，这会导致延迟输出转换过程，并超出所规定的时钟到输出的延迟值(Tco)。亚稳态输出恢复到稳定状态所需的超出 Tco 的额外时间部分称为稳定时间 (Tmet)。

4.1.2 亚稳态产生的原因

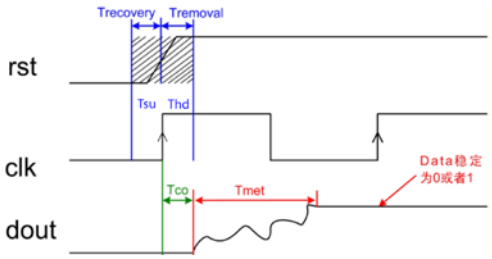
在同步系统中，输入总是与时钟同步，因此寄存器的建立时间和保持时间是满足的，一般情况下是不会发生亚稳态情况的。亚稳态一般是发生在跨时钟域传输和异步信号采集中以及复位电路中：

（1）在跨时钟域传输中，由于时钟之间存在相移，因此当源寄存器发出数据之后，无法确定在什么时间段到达目的寄存器，因此也不能保证满足目的寄存器的建立和保持时间要求。

（2）在异步信号采集当中，由于异步信号可以在任何时间点到达目的寄存器，因此也无法满足目的寄存器的建立和保持时间要求。

（3）复位电路中如果复位信号的撤销正好位于建立和保持时间之间，势必会产生亚稳态。

如图所示，复位信号释放时不满足触发器的建立时间和保持时间，dout 则会出现一段时间的亚稳态，而后随机落于 0 或 1 状态。



4.1.3 亚稳态的处理方法

对亚稳态的情形一般有三种处理方法：

a.对异步信号进行同步处理；

b.采用 FIFO 对跨时钟信号进行缓冲设计；

c.对复位信号采用异步复位、同步处理的方法。

4.2 复位信号的跨时钟域处理

当复位信号在不同时钟域间传递时，由于时钟的异步特性，可能会在目标时钟域里产生亚稳态问题。亚稳态会让触发器的输出在一段时间内处于不确定状态，进而影响系统的稳定性与可靠性，因此需要对其进行处理。

以工程“09_Ti60F225_HDMI_720P_ADCWord_NewVerison”为例，复位信号跨时钟域进行了如下的处理：

（1） 采用同步器，同步化复位信号

```
reg             r_locked = 0;
always @(posedge clk_i or negedge locked_i) begin
    if(~locked_i)
        r_locked <= 0;
    else
        r_locked <= 1;
end
```

这段代码把异步复位信号 locked_i 同步到了目标时钟域 clk，从而输出同步后的复位信号 r_locked。此处仅采用了一级同步器，在对可靠性要求极高的系统中，可采用多级同步器，以此降低亚稳态的影响。

（2） 在同步器后增加复位延迟释放，确保复位信号在目标时钟域稳定释放，避免复位信号过早释放而引发系统不稳定。

```
reg     [15:0]  r_rstn_dly = 0;

always @(posedge clk_i or negedge r_locked) begin
    if(~r_locked) begin
        rstn_o <= 0;
        rst_o <= 1;
        r_rstn_dly <= 0;
    end else begin
        r_rstn_dly <= {r_rstn_dly, 1'b1};
        rstn_o <= r_rstn_dly[15];
        rst_o <= ~r_rstn_dly[15];
    end
end
```

上述代码对 r_locked 实现了延迟 16 个时钟周期的复位释放，确保复位信号在目标时钟域稳定释放。

4.3 其他信号的跨时钟域处理

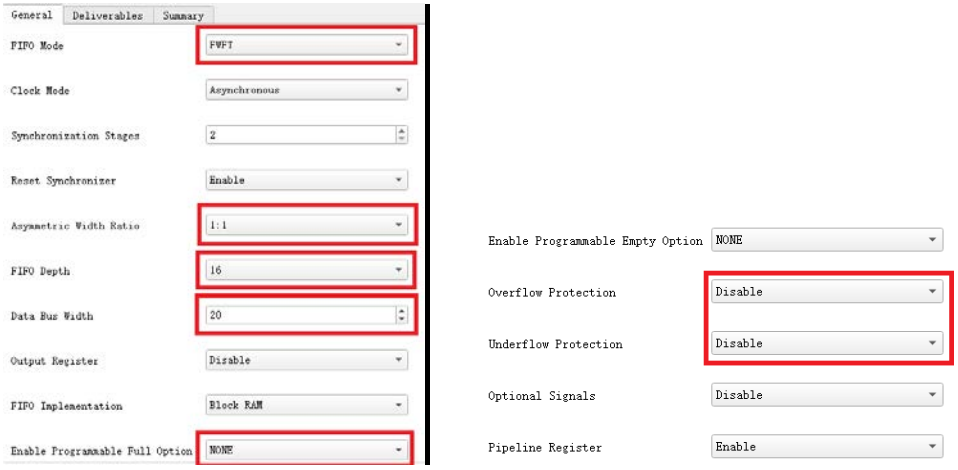
在 FPGA 设计里，常常会涉及到多个时钟域，因此信号的跨时钟域处理十分关键，若处理不当，可能致使系统工作异常，像逻辑错误、数据丢失等状况。由于跨时钟域处理涉及到各种复杂的情况，此处不做进一步介绍，有兴趣的同学可以自主学习。

5 FIFO 使用介绍

FIFO（First Input First Output），先入先出队列，在 FPGA 中主要用于实现数据的有序存储与传输。与 RAM 不同的是，FIFO 无外部地址线，根据内部读写指针自动管理数据，数据地址由内部计数器控制，无法直接通过地址线访问。

工程“09_Ti60F225_HDMI_720P_ADCWord_NewVerison”配置了 2 个 FIFO 的 IP 核，分别为 afifo_w20d16 和 afifo_w8d2048。下面以 afifo_w20d16 为例介绍 FIFO 的使用。

打开 afifo_w20d16 的配置如图所示。FIFO Mode 选择的是 FWFT 模式，在这种模式下数据读取无延迟；Clock Mode 选择 Asynchronous 即异步模式，在此设置下输入输出时钟可以不相同；FIFO Depth 即存储深度为 16 个数据，Data Bus Width 即数据宽度为 20 bit。



观察 FIFO 的接口如下图。wr_en_i 为写入使能信号，当其为高电平时允许 wdata 写入，每个 wr_clk_i 上升沿写入一个数据；rd_en_i 为读出使能信号，当其为高电平时 rdata 有效，每个 rd_clk_i 上升沿读出一个数据；full_o 为高电平时表示 FIFO 中数据已满，此时再写入数据会将 FIFO 中的第 1 个数据覆盖；empty_o 为高电平时表示 FIFO 中无数据，此时 rdata 无效；wr_datacount_o 表示 FIFO 的剩余空间，rd_datacount_o 表示 FIFO 中现有的数据个数。以上信号的含义可在官方的 FIFO 手册中找到。

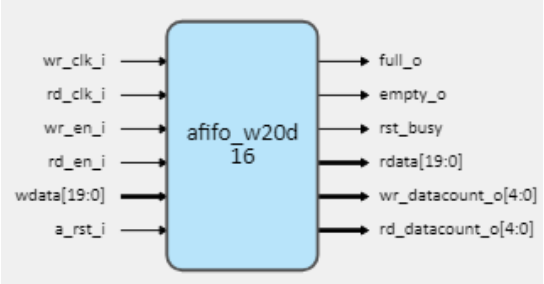


Table 14: FIFO Core Clock, Reset, and Datacount Ports

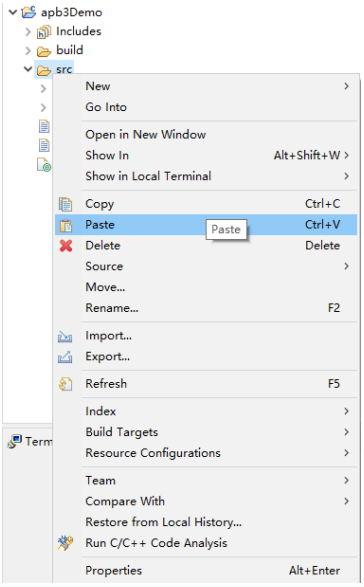
Port	Synchronous	Asynchronous	Direction	Description
a_rst_i	✓	✓	Input	Reset. Asynchronous reset signal that initializes all internal pointers and output flags.
a_wr_rst_i		✓	Input	The incoming reset signal should already synchronized to the write clock domain. This signal is exposed when Reset Synchronizer option is disabled in the IP Configuration window.
a_rd_rst_i		✓	Input	The incoming reset signal should already synchronized to the read clock domain. This signal is exposed when Reset Synchronizer option is disabled in the IP Configuration window.
rst_busy		✓	Output	When asserted, this signal indicate the core is being reset. This signal is asynchronous. You should synchronized it to the respective clock domain before use. This signal is exposed when Reset Synchronizer option is enabled in the IP Configuration window.
wr_clk_i		✓	Input	Write clock. All signals in the write domain are synchronous to this clock.
rd_clk_i		✓	Input	Read clock. All signals in the read domain are synchronous to this clock.
clk_i	✓		Input	Clock. All signals on the write and read domains are synchronous to this clock.
wr_datacount_o [*n*:0]	✓	✓	Output	FIFO write domain data count. Applicable to asymmetric width ratio. $n=\log_2$[DEPTH].
rd_datacount_o [*n*:0]	✓	✓	Output	FIFO read domain data count. Applicable to asymmetric width ratio. $n=\log_2$[DEPTH].
datacount_o [*n*:0]	✓		Output	FIFO data count. Applicable to symmetric width ratio. $n=\log_2$[DEPTH].

Port	Direction	Description
wdata [*m*-1:0]	Input	Write data. The input data bus used when writing to the FIFO buffer. *m*=DATA_WIDTH.
wr_en_i	Input	Write enable. If the FIFO buffer is not full, asserting this signal causes data (on wdata) to be written to the FIFO.
full_o	Output	Full flag. When asserted, this signal indicates that the FIFO buffer is full. Write requests are ignored when the FIFO is full if overflow protection option is enabled. In this case, initiating a write while full is not destructive to the FIFO.
almost_full_o	Output	Optional, almost full. When asserted, this signal indicates that only one more write can be performed before the FIFO is full.
prog_full_o	Output	Optional, programmable full. This signal is asserted when the number of words in the FIFO is greater than or equal to the assert threshold. It is deasserted when the number of words in the FIFO is less than the negate threshold.
wr_ack_o	Output	Optional, write acknowledge. This signal indicates that a write request (wr_en_i) during the prior clock cycle succeeded.
overflow_o	Output	Optional, overflow. This signal is exposed when: **Overflow Protection = Enable** This signal indicates that a write request (wr_en_i) during the prior clock cycle was rejected because the FIFO buffer is full. In this case, overflowing the FIFO is not destructive to the contents of the FIFO. **Optional Signals = Enable** and **Overflow Protection = Disable** This signal indicates that the previous write request overflows the FIFO and it is destructive to the contents of the FIFO. If this signal is asserted, it is required to reset the FIFO.

异步 FIFO 的一个重要作用是缓存数据。当读写数据的速率不相同时，通过 FIFO 可以实现数据缓存，避免数据丢失。与 FIFO 一样，异步 RAM 也可以作缓存数据之用，一般用于图片等数据有明确地址关系的场合，例如工程“V7_Ti60F225_RISCV_APB3_Picture”中的 bram_test。

6 图片取模软件使用介绍

打开 Img2Lcd，左上角打开图片后，选择输出灰度为 24 位真彩色，取消勾选包含图像头数据，点击保存为 C 文件。注意：在例程“V7_Ti60F225_RISCV_APB3_Picture”中，图片大小要求为 60*60，若不符合请先进行裁剪。或者将 Image2Lcd 的最大宽度和高度设置为 60*60，此时需注意实际输出图像大小，如果小于 60*60 则要在生成的文件中补充无效数据使数组长度为 10800。

打开 RISC-V IDE，选择此例程目录下的 workspace。由于路径改动，launch 后需要删除 apb3Demo 工程并重新导入，导入方法可参考《03-基于 Ti60F225 的 RISC-V Demo 搭建流程》。

重新导入 apb3Demo 后，复制刚才生成的.c 文件，右键 src 文件夹将其粘贴进来，修改 main.c 中对应的数组名字即可编译运行。该软件实现将图片数据通过 APB3 写入至 BRAM 中供硬件侧读取显示。